

# Soutenance de thèse

## Conception d'un système mémoire pour traitement de données creuses

Thèse soutenue par

**Valentin ISAAC--CHASSANDE**

*le 11 mai 2026*

Université Grenoble Alpes, CEA List, TiMA

# PLAN

**1.** Les matrices creuses

**2.** Accélérateurs dédiées et Directions

**3.** Cache de réduction

**4.** SpDCache : Fonctionnement et Architecture Matérielle

**5.** Validation de SpDCache via un modèle analytique

**6.** Evaluations et Performances de SpDCache

**7.** Conclusion

# PLAN

## 1. Les matrices creuses

## 2. Accélérateurs dédiés et Directions

## 3. Cache de réduction

## 4. SpDCache : Fonctionnement et Architecture Matérielle

## 5. Validation de SpDCache via un modèle analytique

## 6. Evaluations et Performances de SpDCache

## 7. Conclusion

# LES MATRICES CREUSES

Applications : Physiques numériques, Intelligence artificielle, Analyse de graphes...

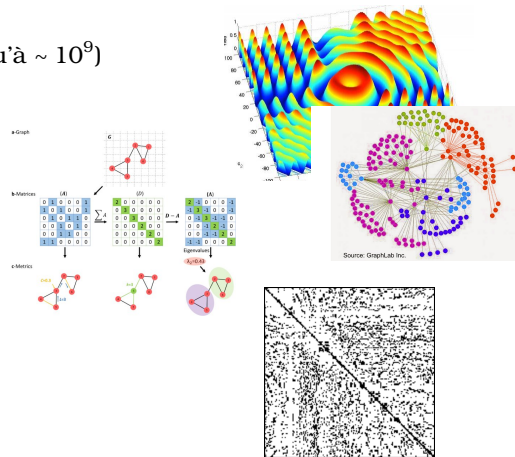
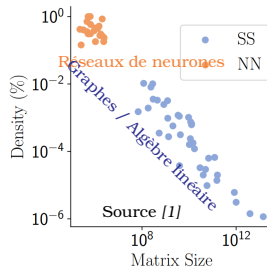
## → Matrice creuse

- **Larges** (nombre d'éléments non-nuls jusqu'à  $\sim 10^9$ )
- **Creuses** (densité jusqu'à  $\sim 10^{-7}$ )

## → Compressées en mémoire : Formats creux

## • Diverses

- En tailles et densités
- En structures



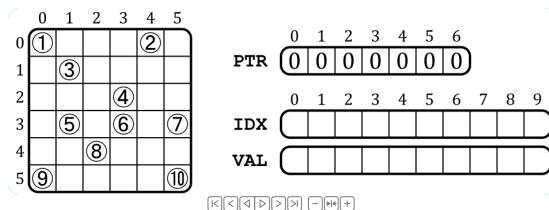
[1] Zhiyao Li, Jiaxiang Li, Taijie Chen, Dimin Niu, Hongzhong Zheng, Yuan Xie, and Mingyu Gao. 2023. Spada : Accelerating Sparse Matrix Multiplication with Adaptive Dataflow. ASPLOS 2023. Association for Computing Machinery, 747–761.

## LES FORMATS CREUX

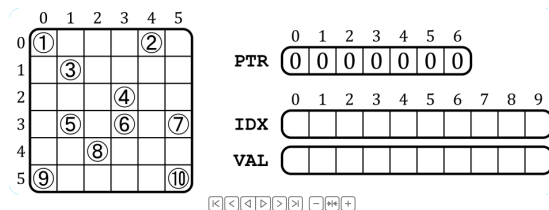
**Formats creux** [2] [3] → pour éviter de stocker les zéros

- Beaucoup de formats spécifiques à des structures de matrices
- CSR et CSC → les plus simples et plus utilisées

### CSR (Compressed Sparse Row)



### CSC (Compressed Sparse Column)



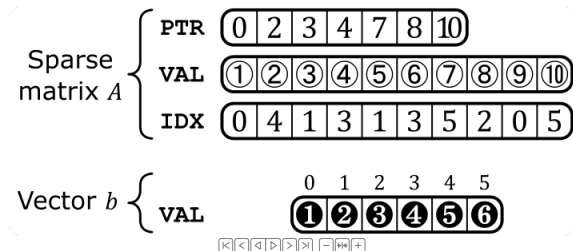
[2] Reginald P. Tewarson. 1973. Sparse matrices. Vol. 69. Academic press New York, State University of New York, Stony Brook, NY.

[3] Yousef Saad. 2003. Iterative Methods for Sparse Linear Systems (second ed.). Society for Industrial and Applied Mathematics.

## ÉCHANGER UN PROBLÈME POUR UN AUTRE

Formats creux :

- **Empreinte mémoire réduite** de  $\mathcal{O}(n^2)$  à  $\mathcal{O}(n)$  → une nécessité
- **Utilisation de métadonnées** → génèrent des transactions mémoires vers des adresses non-consécutives



### Indirections

- **Accès indirects** depuis/vers la hiérarchie mémoire
- Du point de vue d'un système mémoire naïf : **accès « aléatoires »**

## LA HIÉRARCHIE MÉMOIRE

### Pourquoi les accès « aléatoires » sont un problème ?

Noyau de calcul avec une matrice dense → localités spatiale et temporelle  
Noyau de calcul avec une **matrice creuse** → **pas de localité**

Dans une architecture classique

- Les hiérarchies mémoires prennent avantage des accès temporellement et spatialement locaux

Caches → **non-adaptés** aux accès « aléatoires »

- Cache remplie de zéros (lignes de cache remplies à 40% d'éléments non-nuls)
- Nombre de hits bas
- Évincements non-nécessaires → Beaucoup de **transactions mémoires** avec des **temps de réponse élevés**

Simple **préfetcher linéaire** → pas suffisant pour s'occuper du goulot d'étranglement mémoire

## LES NOYAUX DE CACLUL (KERNELS)

Les matrices creuses sont utilisées dans de **nombreux kernels** :

- $1\mathcal{D}$  Multiplication **matrice** creuse-**vecteur** : SpMV
  - $2\mathcal{D}$  Multiplication **matrice-matrice** : SpMSpM, SpGEMM, SpMM
  - $n\mathcal{D}$  Les **tenseurs** en général
- } **Kernels de base**  
utilisés dans la majorité  
des applications

Chaque kernel peut être implémenté suivant **plusieurs algorithmes** :

- **Inner-Product** (IP) ( $\geq 1\mathcal{D}$ )
  - **Outer-Product** (OP) ( $\geq 1\mathcal{D}$ )
  - **Gustavson** (ou Row-wise) ( $\geq 2\mathcal{D}$ )
- } **Algorithmes de base**  
Des algorithmes plus complexes peuvent être vues  
comme des combinaisons de ces trois là.
- Méthodes de tiling → ~ algorithmes customisés / pré-traitement ( $\geq 1\mathcal{D}$ )



# LES INDIRECTIONS DANS LES KERNELS CREUX

## Où apparaissent les indirections ?

**SpMV** :  $A \times b = c$ , avec  $A$  de taille  $(M, N)$

- Dense :
- Creux en Inner-Product (CSR) :
- Creux en Outer-Product (CSC) :

$$c_i += A_{i,j} \times b_j \quad \text{Indirection} \quad (\forall (i,j) \in [0, M-1] \times [0, N-1])$$

$$c_i += VAL_p \times b_{IDX_p} \quad (\forall p \in [PTR_i, PTR_{i+1} - 1])$$

$$c_{IDX_p} += VAL_p \times b_j \quad (\forall p \in [PTR_j, PTR_{j+1} - 1])$$

↑  
Indirection

**SpMM, SpMSpM** :  $A \times B = C$

- Inner-Product : Indirections sur toute la matrice d'entrée  $B$
- Outer-Product : Indirections sur toute la matrice de sortie  $C$
- Gustavson : Indirections sur les lignes de  $B$  et dans les lignes de  $C$ 
  - Intermédiaire entre Inner-Product et Outer-Product

## RÉSUMÉ

**Inner-Product** → accès « aléatoires » sur l'entrée ( $b$  ou  $B$ ) → accès = lecture

→ Problème : comment **rassembler** efficacement les données → **gather**

→ **Intersections** des éléments non-nuls de  $A$  avec ceux de  $b$  ou  $B$  (1D ou 2D)

**Outer-Product** → accès « aléatoires » sur la sortie ( $c$  ou  $C$ ) → accès = lecture + accumulation + écriture

→ Problème : comment **distribuer** efficacement les données → **scatter**

→ **Réductions** des sommes partielles non-nuls dans  $c$  ou  $C$  (1D ou 2D)

**Gustavson** → accès « aléatoires » sur l'entrée et la sortie ( $B$  et  $C$ )

→ Les deux problèmes de **gather** et **scatter** à la fois

→ **Intersections** avec les **lignes de  $B$**  seulement (1D)

→ **Réductions** « aléatoires » dans les **lignes de  $C$**  seulement (1D)

### Problématique initiale

→ Dans le contexte du calcul matriciel, comment adresser efficacement les problématiques d'intersections (gather) et/ou de réductions (scatter) lorsque les données subissent des accès « aléatoires » ?

# PLAN

## 1. Les matrices creuses

## 2. Accélérateurs dédiés et Directions

## 3. Cache de réduction

## 4. SpDCache : Fonctionnement et Architecture Matérielle

## 5. Validation de SpDCache via un modèle analytique

## 6. Evaluations et Performances de SpDCache

## 7. Conclusion

# ACCÉLÉRATEURS MATÉRIELS DÉDIÉS

<flèche temporelle :>

1973 : Approches logicielles sur CPUs [2] / 2005 : Approches logicielles pour GPUs [4] / 2018 - aujourd'hui : **Accélérateurs matériels dédiés au calcul creux**

## Plusieurs caractéristiques

- **Kernel** : Matrice-Matrice   Matrice-Vecteur   Générique
- **Algorithme** : Inner-Product   Outer-Product   Gustavson   Méthodes de tiling
- **Problèmes adressés** : Gather (Intersections)   Scatter (Réductions)
- **Type d'accélérateur** : Complet   Assisté par processeur
- Pour les accélérateurs assistés par processeur, **position dans la hiérarchie mémoire** :  
Proche processeur   Proche caches   Esquive caches   Proche mémoire principale

# ACCÉLÉRATEURS MATÉRIELS DÉDIÉS

<ajouter couleur slide précédente sur table / ajout nb total>

| Nom        | Année | Autonomie |         | Position dans la hiérarchie mémoire |        |      | Algorithme(s) le(s) plus adapté(s) |    |      | Support matériel pour |         | Kernels creux adressés |    |
|------------|-------|-----------|---------|-------------------------------------|--------|------|------------------------------------|----|------|-----------------------|---------|------------------------|----|
|            |       | Complet   | Assisté | Proc.                               | Caches | Mém. | IP                                 | OP | Gust | Gather                | Scatter | MV                     | MM |
| Coup       | 2015  |           | ✓       |                                     | ✓      |      |                                    | ✓  |      |                       | ✓       | ✓                      | ~  |
| EIE        | 2016  | ✓         |         |                                     |        |      |                                    | ✓  |      | ✓                     | ✓       | ✓                      |    |
| OuterSPACE | 2018  | ✓         |         |                                     |        |      |                                    | ✓  |      | ✓                     | ✓       | ✓                      | ✓  |
| GraphR     | 2018  | ✓         |         |                                     |        |      |                                    | ✓  |      | ✓                     | ✓       | ✓                      |    |
| SDE        | 2019  | ✓         |         |                                     |        |      | ✓                                  |    | ~    | ✓                     | ~       | ✓                      | ~  |
| ExTensor   | 2019  | ✓         |         |                                     |        |      | ✓                                  | ~  |      | ✓                     | ~       | ~                      | ✓  |
| PHI        | 2019  |           | ✓       |                                     | ✓      |      |                                    | ✓  | ~    |                       | ✓       | ✓                      | ~  |
| SPiDRE     | 2019  |           | ✓       |                                     |        | ✓    | ✓                                  |    |      | ✓                     |         | ✓                      | ~  |
| ITS        | 2019  | ✓         |         |                                     |        |      |                                    | ✓  |      | ✓                     | ✓       | ✓                      |    |
| SPU        | 2019  | ✓         |         |                                     |        |      | ✓                                  | ✓  |      | ✓                     | ✓       | ✓                      |    |
| Tensaurus  | 2020  | ✓         |         |                                     |        |      | ~                                  | ~  |      | ✓                     | ✓       | ✓                      | ✓  |
| MatRaptor  | 2020  | ✓         |         |                                     |        |      |                                    |    | ✓    | ✓                     | ✓       |                        | ✓  |
| SpArch     | 2020  | ✓         |         |                                     |        |      |                                    | ✓  |      | ✓                     | ✓       |                        | ✓  |

→ **Diversité** des choix de design, toutes les caractéristiques sont couvertes

Exploration des algorithmes **IP et OP**

Exploration de l'algorithme de **Gustavson**

**Combinaisons d'algorithmes**

## DESIGN SPACE

<diagramme complet - simplifié?>

Montrer liens assist = générique, ... -> différences entre complets et assistés (avantages et inconvénients)

## QUELLE EST LA MEILLEURE SOLUTION ?

Les accélérateurs matériels dédiés

- **Répondent efficacement à la problématique du calcul creux** ✓

- Les méthodes et choix de design sont **diverses**

→ Une comparaison de performance exhaustive est difficile

→ **Une comparaison juste est difficile à réaliser**

→ À la place, on a une **vision claire du design space**

→ On compare seulement les performances des algorithmes pour les accélérateurs complets

## UN BESOIN DE SUPPORT POUR LES RÉDUCTIONS

Parmi les accélérateurs complets (majoritairement pour les kernels Matrice-Matrice)

→ **N°1 : Gustavson** présente les meilleurs performances et avantages

→ **N°2 : Outer-Product**, N°1 pour le kernel SpMV

Vérifiable avec une **analyse gros grain** :

→ Nombre de données transférées en mémoire principale

→ Pour chaque algorithme, en fonction d'une taille de mémoire locale

→ **Besoin de support pour les réductions « aléatoires »**

En général, pour tous les accélérateurs :

→ Grandes **variations de performances** selon les matrices et leurs **structures**

→ **Besoin de considérer les structures des matrices**



## DIRECTION DE CETTE THÈSE

### Hypothèses :

- Noyau de calcul : **SpMV** (multiplication matrice creuse-vecteur)
- Algorithme : **OP** (*Outer-Product*)
- Problème : Scatter → **Réductions « aléatoires »**
- Architecture de **type générique**, faible coût en surface  
→ Au niveau des **caches de données**

### Approche :

- **SpDCache** : Cache de données spécialisé qui adresse les réductions « aléatoires »

**Objectif 1 :** Mitiger l'irrégularité des réductions en analysant le comportement des caches de données

**Objectif 2 :** Comprendre le rôle des structures des matrices dans les performances

# PLAN

## 1. Les matrices creuses

## 2. Accélérateurs dédiées et Directions

## 3. Cache de réduction

- Cache générique
- Cache de réduction

## 4. SpDCache : Fonctionnement et Architecture Matérielle

## 5. Validation de SpDCache via un modèle analytique

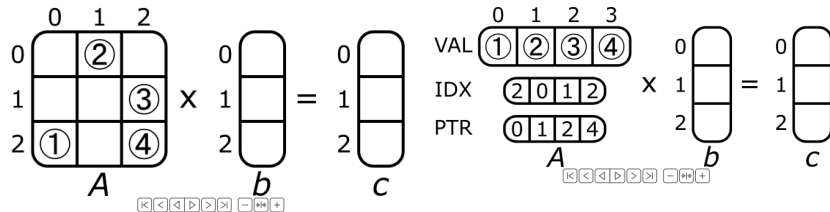
## 6. Evaluations et Performances de SpDCache

## 7. Conclusion

## SPMV EN OUTER-PRODUCT

Exemple de **SpMV** :  $A \times b = c$ , avec  $A$  de taille  $(M, N)$ .

- $\forall (i, j) \in \llbracket 0, M-1 \rrbracket \times \llbracket 0, N-1 \rrbracket$ ,  $c_i = c_i + A_{i,j} \times b_j$
- $A$  est stockée au format creux CSC ( $VAL$ ,  $IDX$ ,  $PTR$ )
- **Outer-Product (OP)** : des **accès indirectes** apparaissent sur la sortie



Indirections

$$\begin{aligned}
 c_{IDX_0} &+= VAL_0 \times b_0 \quad \textcircled{1} \\
 c_{IDX_1} &+= VAL_1 \times b_1 \quad \textcircled{2} \\
 c_{IDX_2} &+= VAL_2 \times b_2 \quad \textcircled{3} \\
 c_{IDX_3} &+= VAL_3 \times b_2 \quad \textcircled{4}
 \end{aligned}$$

→ Avec le **SpMV** en **Outer-Product**, les accès « aléatoires » se produisent sur le vecteur de sortie **c**, au moment d'accumuler la somme partielle  $A_{i,j} \times b_j$  avec  $c_i$ .

→ **Réductions « aléatoires »**

→ Lectures séquentiels des matrice et vecteur d'entrées  $A$  et  $b$

# RÉDUCTIONS

## Qu'est-ce qu'une réduction ?

Une réduction sur le vecteur  $c$  :  $c_{indice} += valeur$

- Une **lecture** en mémoire de  $c_{indice}$
- Une **accumulation** pour mettre à jour  $c_{indice}$  avec  $valeur$
- Une **écriture** en mémoire de  $c_{indice}$  mis à jour

Cette opération est noté **RED** :  $RED(indice, valeur)$

**Cache générique** : pas de support particulier pour les opérations de réduction

→ **Cache de réduction** (ex. : [5] [6])

- Supporte une **nouvelle instruction RED** :  $RED(adresse, valeur)$
- L'opération de réduction est **traitée dans le cache**
  - Possède une **unité de calcul**

[5] A. Mukkara, N. Beckmann, and D. Sanchez. 2019. PHI : Architectural Support for Synchronization- and Bandwidth-Efficient Commutative Scatter Updates. In Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO-52). Association for Computing Machinery, New York, NY, USA, 1009–1022.

[6] S. Srikanth, A. Jain, T. M. Conte, E. P. DeBenedictis, and J. Cook. 2021. SortCache : Intelligent Cache Management for Accelerating Sparse Data Workloads. ACM Trans. Archit. Code Optim. 18, 4, Article 56 (sep 2021), 24 pages.

# CACHE GÉNÉRIQUE

## Comment se comporte une réduction dans un cache générique ?

### • Cas de hit :

- Lecture du cache ✓
- Accumulation dans le processeur
- Ecriture dans le cache ✓

### • Cas de miss :

- Lecture en mémoire et mise à jour du cache ✗
- Accumulation dans le processeur
- Ecriture dans le cache ✓

### • Cas de miss avec évincement :

- Ecriture en mémoire de la donnée évincée ✗
- Lecture en mémoire et mise à jour du cache ✗
- Accumulation dans le processeur
- Ecriture dans le cache ✓



<schéma d'une réduction dans un cache générique>

## CACHE DE RÉDUCTION



- **Nombre intrinsèque de transactions mémoires**
  - Cache générique :  $0 + 1 + 2 = 3$  ✗
  - Cache de réduction :  $0 + 0 + 2 = 2$  ✓
- Les réductions sont « **envoyées et oubliées** » :
  - Le processeur **n'attend pas de réponse**, même à l'évincement

**SpDCache : cache de réduction**

→ **Reductions** ✓ « **aléatoires** » ✗

# PLAN

## 1. Les matrices creuses

## 2. Accélérateurs dédiés et Directions

## 3. Cache de réduction

## 4. SpDCache : Fonctionnement et Architecture Matérielle

## 5. Validation de SpDCache via un modèle analytique

## 6. Evaluations et Performances de SpDCache

## 7. Conclusion

## LES MATRICES BANDÉES

SpDCache : s'inspire d'une structure de matrice très répandue, la structure bandée

<Présenter les matrices bandées>

SpDCache et les matrices bandées ->



## CONCEPT DE SPDCACHE

Par rapport à un cache générique :

- Ajout du **mode réduction**
- Ajout d'une **FPU** (unité de calcul flottant)
- Partage de la mémoire cache en **trois régions** :

- **DRC** pour la région dense, en-bande
  - Vecteur  $c$  uniquement
  - Réductions uniquement
  - Granularité d'une ligne de cache
  - Associatif à ensemble
- **SRT** pour la région creuse, hors-bande
  - Vecteur  $c$  uniquement
  - Réductions uniquement
  - Granularité d'un élément
  - Complètement associatif
- **GRC**
  - Lectures génériques de  $A$  et  $b$

<ajouter figure avec les modifs apportés à un cache générique>

## ARCHITECTURE DE SPDcACHE

Du texte pour expliquer comment passent les données dans le cache

A, b dans GRC

Dans processeur : En ou hors bande de A? /

Axb -> c

RED (c Axb reg) vers L1

Selon reg, DRC ou SRT

Evincements vers mémoire, soit ligne wise soit RMW

► Utilise des transactions atomique RMW  
(lecture-accumulation-écriture)

<figure de spdcache + direction des transactions (globales) de A,b à c>

# ARCHITECTURE DE SPDcACHE

Transactions RED -> "envoyées et oubliées"

<figure avec animations du fonctionnement de  
spdcache>

Hit / Miss

- **Pas** de transaction mémoire
- Insertion ou accumulation dans le cache

Miss avec évincement dans le DRC

- 2 transactions mémoires
- Accumulation dans le cache

Miss avec évincement dans la SRT

- 1 transaction mémoire
- "envoyé et oublié"
- Accumulation en mémoire principale

(note sur les duplications)

# PLAN

1. Les matrices creuses

2. Accélérateurs dédiées et Directions

3. Cache de réduction

4. SpDCache : Fonctionnement et Architecture Matérielle

**5. Validation de SpDCache via un modèle analytique**

6. Evaluations et Performances de SpDCache

7. Conclusion

## MULTIPLICATION MATRICE-VECTEUR DANS UN CACHE GÉNÉRIQUE

Matrice quelconque :

- Évincements répétés dans le cache
- Le cache n'est pas utile ✖

Petite matrice, bande horizontale :

- Tiens dans le cache, pas d'évincement
- Le cache est utile ✔
- Cas peu intéressants ✖

Matrice bandée :

- Évincements répétés dans le cache
- Le cache n'est pas utile ✖
- Structure très courante ✔

Matrice parfaitement bandée :

- Nombre d'évincements limité
- Le cache est utile ✔
- Structure peu courante ✖



list

# TRAFFIC MÉMOIRE THÉORIQUE AVEC DES MATRICES BANDÉE

C

Traffic mémoire du vecteur de sortie  
*versus*

densité en dehors de la bande,  
pour différentes tailles de cache

- Densité dans la bande fixe ( $10^{-2}$ )
  -
- Réduction de deux ordres de grandeur
- SI Matrice parfaitement bandée
  - ET Largeur de bande suffisamment petite par rapport à la taille du cache

<relation>



list

(graph avec courbes qui apparaissent au bon moment)

## CONCLUSION

On sélectionne une bande diagonale de la matrice, qu'on traite avec un cache set-associatif

- <relation>

## QUE FAIRE DU RESTE, HORS DE LA BANDE ?

Granularité : Éléments isolés

- Incompatible avec les lignes de cache

→ Pas de ligne de cache :

64o (8 mots) → 8o (1 mot)

Absence de structure claire

- Nombre de *hits* bas

→ Pour augmenter la quantité de *hits*  
dans un espace de cache modeste :

Cache complètement associatif

Transactions mémoires inadaptées

- Lectures et écritures sur 64o (8 mots)

→ Transactions mémoires "par éléments"

RMW atomique sur 8o (1 mot)



(graph avec courbes qui apparaissent au bon moment)

### CONCLUSION

On traite les éléments hors-bande de la matrice avec un cache « par élément », complètement associatif, avec des transactions mémoires « par élément » (RMW)

## CONCLUSION

Conclusion un peu globale : SpDCache permet d'adresser le problème des réductions aléatoires en utilisant intelligemment l'espace mémoire disponible ("bloquer" la région la plus dense / gérer les éléments isolés)



# PLAN

## 1. Les matrices creuses

## 2. Accélérateurs dédiés et Directions

## 3. Cache de réduction

## 4. SpDCCache : Fonctionnement et Architecture Matérielle

## 5. Validation de SpDCCache via un modèle analytique

## 6. Evaluations et Performances de SpDCCache

## 7. Conclusion

## ENVIRONNEMENT

CVA6 + SpDCache + RAM

## TRAFFIC MÉMOIRE

Set de 48 à 62 matrices

Toutes structures

Config 25/50/25

Perfs avec écart type (brut et ×)

Conclu : réduction significative du trafic mémoire

On note deux groupes (transition vers speedup)

## ACCÉLÉRATION

1 coeur CVA6, basse perf -> on est dans une zone entre compute et memory bound

Permet de clasifier les marices selon nouveau critere dcd

Explique les deux groupes d'accélération

Conclu : Comme attendu, reduction du trafic permet accélération sur matrice memory bound /  
classification des matrices selon densité des lignes de cache

# PLAN

## 1. Les matrices creuses

## 2. Accélérateurs dédiées et Directions

## 3. Cache de réduction

## 4. SpDCCache : Fonctionnement et Architecture Matérielle

## 5. Validation de SpDCCache via un modèle analytique

## 6. Evaluations et Performances de SpDCCache

## 7. Conclusion

## CONCLUSION

Revenir sur la problématique  
Et sur les Q1 et Q2

slide publi et autres ?

*Thanks!*

contact



backup slides :

Exploration des paramètres (Python)

BaCSC

Toutes les perfs

Perf Python

Algo Gust SpMSpM

Analyses théoriques des algos de SpMSpM / SpMV

1 ou 2 tableaux comparant les algos de façon détaillé (voir manuscrit)

Ajouter exemple d'accélérateurs du SoA ? (Gamma et PHI)

Quelques slides sur le RTL

Les comparaisons de modèles

Les autres graphes de l'analyse théorique

Algo IP et OP en creux